

# Cycle de vie d'une application Android

## Table des matières

|                         |   |
|-------------------------|---|
| Exemple à étudier ..... | 2 |
| Journalisation .....    | 3 |
| Cycle de vie .....      | 4 |
| Références .....        | 6 |

## Exemple à étudier

```
1 private const val TAG = "LOG MainActivity"
2
3 class MainActivity : ComponentActivity() {
4     override fun onCreate(savedInstanceState: Bundle?) {
5         super.onCreate(savedInstanceState)
6         Log.d(TAG, "onCreate()")
7         setContent {
8             CycleDeVieTheme {
9                 Surface(
10                     modifier = Modifier.fillMaxSize(),
11                     color = MaterialTheme.colorScheme.background
12                 ) {
13                     JeuClick()
14                 }
15             }
16         }
17     }
18     override fun onStart() {
19         super.onStart()
20         Log.d(TAG, "onStart()")
21     }
22
23     override fun onResume() {
24         super.onResume()
25         Log.d(TAG, "onResume()")
26     }
27
28     override fun onRestart() {
29         super.onRestart()
30         Log.d(TAG, "onRestart()")
31     }
32
33     override fun onPause() {
34         super.onPause()
35         Log.d(TAG, "onPause()")
36     }
37
38     override fun onStop() {
39         super.onStop()
40         Log.d(TAG, "onStop()")
41     }
42
43     override fun onDestroy() {
44         super.onDestroy()
45         Log.d(TAG, "onDestroy()")
46     }
47 }
48
49 @Composable
50 fun JeuClick(modifier: Modifier = Modifier) {
51     var valeur = rememberSaveable { mutableStateOf(0) }
```

```

52     Column (
53         modifier = modifier,
54         horizontalAlignment = Alignment.CenterHorizontally
55     ) {
56         Spacer(modifier = Modifier.height(64.dp))
57         Text(
58             text = valeur.value.toString(),
59             modifier = modifier
60         )
61         Spacer(modifier = Modifier.height(16.dp))
62         Button(onClick = { /*valeur.value = (1..6).random()*valeur.value += 1
    }) {
63             Text("Click")
64         }
65     }
66 }
67
68 @Preview(showBackground = true)
69 @Composable
70 fun GreetingPreview() {
71     CycleDeVieTheme {
72         JeuClick()
73     }
74 }

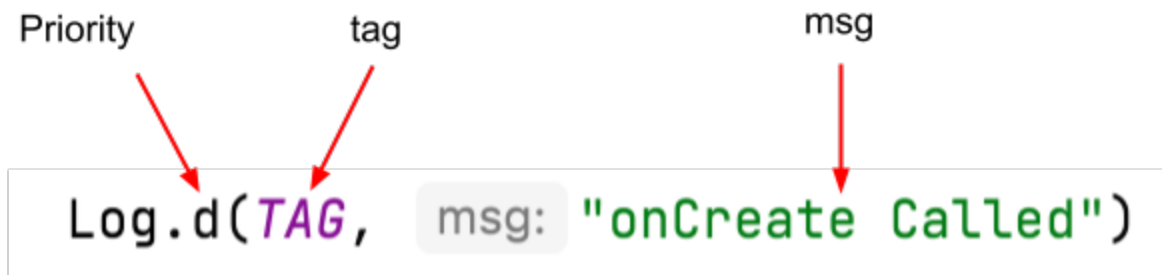
```

## Journalisation

La classe **Log** écrit les messages dans l'outil **Logcat**. **Logcat** est la console de journalisation des messages. C'est là que les messages Android concernant votre application s'affichent, y compris ceux que vous envoyez explicitement au journal avec la méthode **Log.d()** ou d'autres méthodes de classe **Log**.

L'instruction **Log** repose sur trois aspects importants :

- La **priorité** du message de journal, c'est-à-dire son importance. Dans ce cas, **Log.v()** consigne des messages détaillés. La méthode **Log.d()** écrit un message de débogage. Les autres méthodes de la classe **Log** incluent **Log.i()** pour les messages d'information, **Log.w()** pour les avertissements et **Log.e()** pour les messages d'erreur.
- Le paramètre **tag** est une chaîne de caractère qui permet de reconnaître plus facilement les messages dans le journal. Généralement le nom de la classe est utilisé.
- Le paramètre **msg** est une chaîne de caractères courte qui contient le message à afficher.



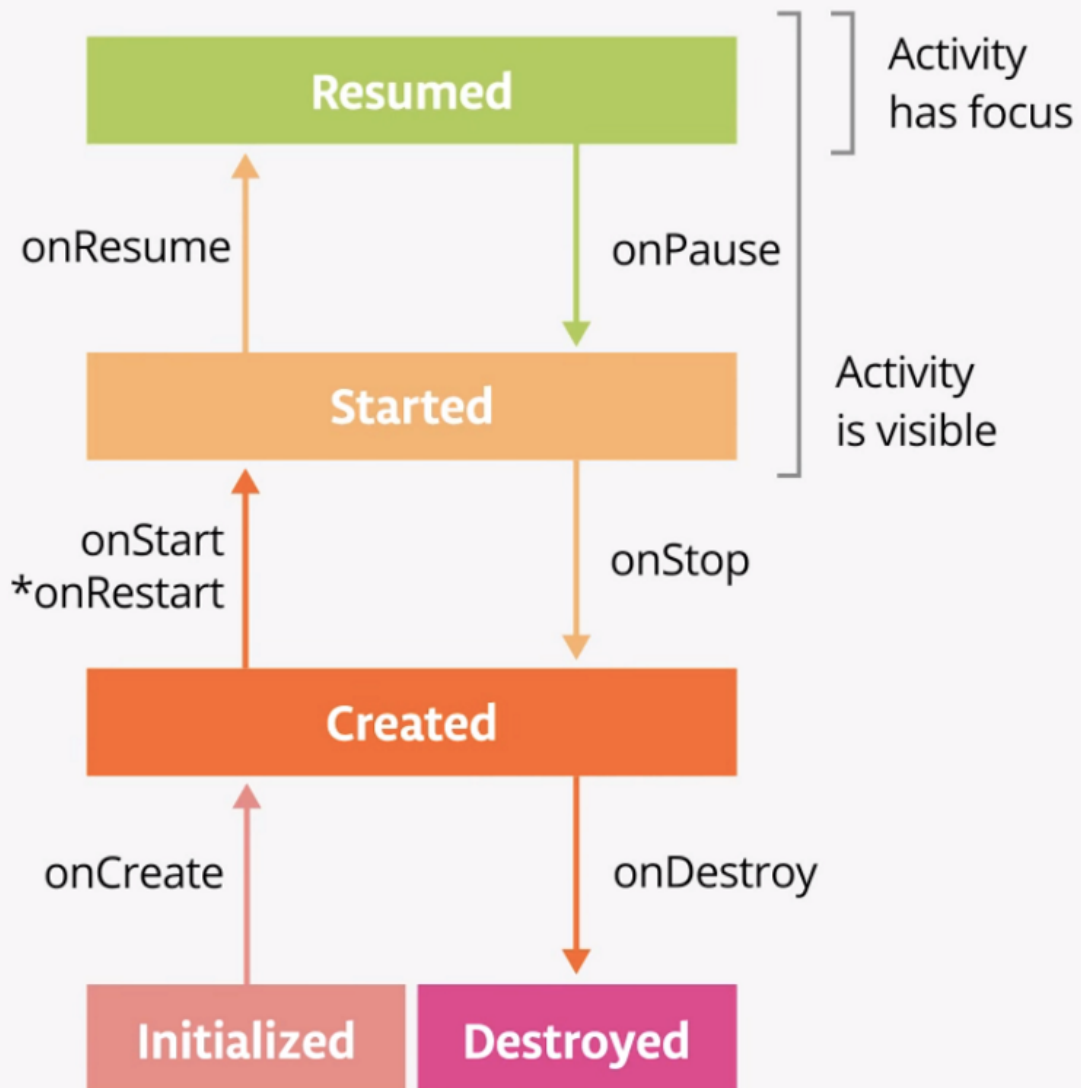
À la ligne x, de l'exemple précédent, une constante globale `TAG` est déclarée. La méthode `Log.d()` est utilisée à la ligne y.

## Cycle de vie

Chaque application a ce que l'on appelle un *cycle de vie*. Le cycle de vie d'une application est constitué des différents états par lesquels une application peut passer, de sa première initialisation à sa destruction. En règle générale, le point d'entrée d'un programme est la méthode `main()`. Cependant, les applications Android commencent par la méthode `onCreate()`. Comme vous avez déjà utilisé des activités à de nombreuses reprises tout au long de ce cours, il se peut que vous reconnaissiez la méthode `onCreate()`.

Le diagramme suivant illustre tous les états du cycle de vie d'une application. Comme leur nom l'indique, ces états représentent le statut de l'application.

# The Activity Lifecycle



Dans l'exemple précédent, des méthodes `Log.d()` ont été ajoutées aux méthodes `onCreate()`, `onStart()`, `onResume()`...

Examiner le journal après quelques manipulations de l'application.

# Références

- [Étape du cycle de vie d'une activité](#)

1 **Mettre** le code ici